`.Receptionist.Listing`. The adapter needs to wrap this message in the following protocol, which belongs to the concierge:

```
sealed trait Command
private final case class ListingResponse(listing: Receptionist.Listing)
    extends Command
```

Because `ListingResponse` contains only a `Receptionist.Listing`, you can directly use the `context.messageAdapter` to map the two:

```
val listingNotificationAdapter =
      context.messageAdapter[Receptionist.Listing](ListingResponse)
```

With the adapter, the concierge can now subscribe by sending a `Subscribe` message with the `goldenKey`:

```
context.system.receptionist ! Receptionist
  .Subscribe(goldenKey, listingNotificationAdapter)
```

Finally, when the concierge receives the registration list, it logs each guest's name. This happens every time a guest registers. You can see the whole actor in the following listing.

**Listing 6.1 `HotelConcierge` actor**

```
import akka.actor.typed.scaladsl.{ Behaviors }
import akka.actor.typed.receptionist.{ Receptionist, ServiceKey }

object HotelConcierge {

  val goldenKey = ServiceKey[VIPGuest.Command]("concierge-key")          ❶

  sealed trait Command
  private final case class ListingResponse(listing: Receptionist.Listing) ❷
      extends Command
```